7.4 Iterators

An ***iterator*** is a software design pattern that abstracts the process of scanning through a sequence of elements, one element at a time. The underlying elements might be stored in a container class, streaming through a network, or generated by a series of computations.

In order to unify the treatment and syntax for iterating objects in a way that is independent from a specific organization, Java defines the java.util.Iterator interface with the following two methods:

hasNext( ): Returns true if there is at least one additional element in the sequence, and false otherwise.

next( ): Returns the next element in the sequence.

The interface uses Java's generic framework, with the next( ) method returning a parameterized element type. For example, the Scanner class (described in Section 1.6) formally implements the Iterator<String> interface, with its next( ) method returning a String instance.

If the next( ) method of an iterator is called when no further elements are available, a NoSuchElementException is thrown. Of course, the hasNext( ) method can be used to detect that condition before calling next( ).

The combination of these two methods allows a general loop construct for processing elements of the iterator. For example, if we let variable, iter, denote an instance of the Iterator<String> type, then we can write the following:

```
while (iter.hasNext( )) {
  String value = iter.next( );
  System.out.println(value);
}
```

The java.util.Iterator interface contains a third method, which is *optionally* supported by some iterators:

remove( ): Removes from the collection the element returned by the most recent call to next( ). Throws an IllegalStateException if next has not yet been called, or if remove was already called since the most recent call to next.

This method can be used to filter a collection of elements, for example to discard all negative numbers from a data set.

For the sake of simplicity, we will not implement the remove method for most data structures in this book, but we will give two tangible examples later in this section. If removal is not supported, an UnsupportedOperationException is conventionally thrown.

www.EBooksWorld.ir

7.4.1 The Iterable Interface and Java's For-Each Loop

A single iterator instance supports only one pass through a collection; calls to next can be made until all elements have been reported, but there is no way to "reset" the iterator back to the beginning of the sequence.

However, a data structure that wishes to allow repeated iterations can support a method that returns a *new* iterator, each time it is called. To provide greater standardization, Java defines another parameterized interface, named Iterable, that includes the following single method:

iterator( ): Returns an iterator of the elements in the collection.

An instance of a typical collection class in Java, such as an ArrayList, is ***iterable*** (but not itself an ***iterator***); it produces an iterator for its collection as the return value of the iterator( ) method. Each call to iterator( ) returns a new iterator instance, thereby allowing multiple (even simultaneous) traversals of a collection.

Java's Iterable class also plays a fundamental role in support of the "for-each" loop syntax (described in Section 1.5.2). The loop syntax,

```
for (ElementType variable : collection) {
  loopBody                                    // may refer to "variable"
}
```

is supported for any instance, *collection*, of an iterable class. *ElementType* must be the type of object returned by its iterator, and *variable* will take on element values within the *loopBody*. Essentially, this syntax is shorthand for the following:

```
Iterator<ElementType> iter = collection.iterator( );
while (iter.hasNext( )) {
  ElementType variable = iter.next( );
  loopBody                                    // may refer to "variable"
}
```

We note that the iterator's remove method cannot be invoked when using the for-each loop syntax. Instead, we must explicitly use an iterator. As an example, the following loop can be used to remove all negative numbers from an ArrayList of floating-point values.

```
ArrayList<Double> data;  // populate with random numbers (not shown)
Iterator<Double> walk = data.iterator( );
while (walk.hasNext( ))
  if (walk.next( ) < 0.0)
    walk.remove( );
```

www.EBooksWorld.ir

7.4.2 Implementing Iterators

There are two general styles for implementing iterators that differ in terms of what work is done when the iterator instance is first created, and what work is done each time the iterator is advanced with a call to next( ).

A ***snapshot iterator*** maintains its own private copy of the sequence of elements, which is constructed at the time the iterator object is created. It effectively records a "snapshot" of the sequence of elements at the time the iterator is created, and is therefore unaffected by any subsequent changes to the primary collection that may occur. Implementing snapshot iterators tends to be very easy, as it requires a simple traversal of the primary structure. The downside of this style of iterator is that it requires $O(n)$ time and $O(n)$ auxiliary space, upon construction, to copy and store a collection of n elements.

A ***lazy iterator*** is one that does not make an upfront copy, instead performing a piecewise traversal of the primary structure only when the next( ) method is called to request another element. The advantage of this style of iterator is that it can typically be implemented so the iterator requires only $O(1)$ space and $O(1)$ construction time. One downside (or feature) of a lazy iterator is that its behavior is affected if the primary structure is modified (by means other than by the iterator's own remove method) before the iteration completes. Many of the iterators in Java's libraries implement a "fail-fast" behavior that immediately invalidates such an iterator if its underlying collection is modified unexpectedly.

We will demonstrate how to implement iterators for both the ArrayList and LinkedPositionalList classes as examples. We implement lazy iterators for both, including support for the remove operation (but without any fail-fast guarantee).

Iterations with the ArrayList class

We begin by discussing iteration for the ArrayList<E> class. We will have it implement the Iterable<E> interface. (In fact, that requirement is already part of Java's List interface.) Therefore, we must add an iterator( ) method to that class definition, which returns an instance of an object that implements the Iterator<E> interface. For this purpose, we define a new class, ArrayIterator, as a nonstatic nested class of ArrayList (i.e., an ***inner class***, as described in Section 2.6). The advantage of having the iterator as an inner class is that it can access private fields (such as the array A) that are members of the containing list.

Our implementation is given in Code Fragment 7.13. The iterator( ) method of ArrayList returns a new instance of the inner ArrayIterator class. Each iterator maintains a field j that represents the index of the next element to be returned. It is initialized to 0, and when j reaches the size of the list, there are no more elements to return. In order to support element removal through the iterator, we also maintain a boolean variable that denotes whether a call to remove is currently permissible.

www.EBooksWorld.ir

```
1   //---------- nested ArrayIterator class ----------
2   /**
3    * A (nonstatic) inner class. Note well that each instance contains an implicit
4    * reference to the containing list, allowing it to access the list's members.
5    */
6   private class ArrayIterator implements Iterator<E> {
7     private int j = 0;                    // index of the next element to report
8     private boolean removable = false;    // can remove be called at this time?
9
10    /**
11     * Tests whether the iterator has a next object.
12     * @return true if there are further objects, false otherwise
13     */
14    public boolean hasNext( ) { return j < size; }  // size is field of outer instance
15
16    /**
17     * Returns the next object in the iterator.
18     *
19     * @return next object
20     * @throws NoSuchElementException if there are no further elements
21     */
22    public E next( ) throws NoSuchElementException {
23      if (j == size) throw new NoSuchElementException("No next element");
24      removable = true;    // this element can subsequently be removed
25      return data[j++];    // post-increment j, so it is ready for future call to next
26    }
27
28    /**
29     * Removes the element returned by most recent call to next.
30     * @throws IllegalStateException if next has not yet been called
31     * @throws IllegalStateException if remove was already called since recent next
32     */
33    public void remove( ) throws IllegalStateException {
34      if (!removable) throw new IllegalStateException("nothing to remove");
35      ArrayList.this.remove(j-1);      // that was the last one returned
36      j--;                             // next element has shifted one cell to the left
37      removable = false;               // do not allow remove again until next is called
38    }
39  } //---------- end of nested ArrayIterator class ----------
40
41  /** Returns an iterator of the elements stored in the list. */
42  public Iterator<E> iterator( ) {
43    return new ArrayIterator( );        // create a new instance of the inner class
44  }
```

Code Fragment 7.13: Code providing support for ArrayList iterators. (This should be nested within the ArrayList class definition of Code Fragments 7.2 and 7.3.)

www.EBooksWorld.ir

Iterations with the LinkedPositionalList class

In support the concept of iteration with the LinkedPositionalList class, a first question is whether to support iteration of the *elements* of the list or the *positions* of the list. If we allow a user to iterate through all positions of the list, those positions could be used to access the underlying elements, so support for position iteration is more general. However, it is more standard for a container class to support iteration of the core elements, by default, so that the for-each loop syntax could be used to write code such as the following,

```
for (String guest : waitlist)
```

assuming that variable waitlist has type LinkedPositionalList<String>.

For maximum convenience, we will support *both* forms of iteration. We will have the standard iterator( ) method return an iterator of the elements of the list, so that our list class formally implements the Iterable interface for the declared element type.

For those wishing to iterate through the positions of a list, we will provide a new method, positions( ). At first glance, it would seem a natural choice for such a method to return an Iterator. However, we prefer for the return type of that method to be an instance that is Iterable (and hence, has its own iterator( ) method that returns an iterator of positions). Our reason for the extra layer of complexity is that we wish for users of our class to be able to use a for-each loop with a simple syntax such as the following:

```
for (Position<String> p : waitlist.positions( ))
```

For this syntax to be legal, the return type of positions( ) must be Iterable.

Code Fragment 7.14 presents our new support for the iteration of positions and elements of a LinkedPositionalList. We define three new inner classes. The first of these is PositionIterator, providing the core functionality of our list iterations. Whereas the array list iterator maintained the index of the next element to be returned as a field, this class maintains the position of the next element to be returned (as well as the position of the most recently returned element, to support removal).

To support our goal of the positions( ) method returning an Iterable object, we define a trivial PositionIterable inner class, which simply constructs and returns a new PositionIterator object each time its iterator( ) method is called. The positions( ) method of the top-level class returns a new PositionIterable instance. Our framework relies heavily on these being inner classes, not static nested classes.

Finally, we wish to have the top-level iterator( ) method return an iterator of elements (not positions). Rather than reinvent the wheel, we trivially adapt the PositionIterator class to define a new ElementIterator class, which lazily manages a position iterator instance, while returning the element stored at each position when next( ) is called.

www.EBooksWorld.ir

```
1   //---------------- nested PositionIterator class ----------------
2   private class PositionIterator implements Iterator<Position<E>> {
3     private Position<E> cursor = first();    // position of the next element to report
4     private Position<E> recent = null;       // position of last reported element
5     /** Tests whether the iterator has a next object. */
6     public boolean hasNext() { return (cursor != null);    }
7     /** Returns the next position in the iterator. */
8     public Position<E> next() throws NoSuchElementException {
9       if (cursor == null) throw new NoSuchElementException("nothing left");
10      recent = cursor;                // element at this position might later be removed
11      cursor = after(cursor);
12      return recent;
13    }
14    /** Removes the element returned by most recent call to next. */
15    public void remove() throws IllegalStateException {
16      if (recent == null) throw new IllegalStateException("nothing to remove");
17      LinkedPositionalList.this.remove(recent);                // remove from outer list
18      recent = null;                  // do not allow remove again until next is called
19    }
20  } //------------ end of nested PositionIterator class ------------
21
22  //---------------- nested PositionIterable class ----------------
23  private class PositionIterable implements Iterable<Position<E>> {
24    public Iterator<Position<E>> iterator() { return new PositionIterator(); }
25  } //------------ end of nested PositionIterable class ------------
26
27  /** Returns an iterable representation of the list's positions. */
28  public Iterable<Position<E>> positions() {
29    return new PositionIterable();          // create a new instance of the inner class
30  }
31
32  //---------------- nested ElementIterator class ----------------
33  /* This class adapts the iteration produced by positions() to return elements. */
34  private class ElementIterator implements Iterator<E> {
35    Iterator<Position<E>> posIterator = new PositionIterator();
36    public boolean hasNext() { return posIterator.hasNext(); }
37    public E next() { return posIterator.next().getElement(); } // return element!
38    public void remove() { posIterator.remove(); }
39  }
40
41  /** Returns an iterator of the elements stored in the list. */
42  public Iterator<E> iterator() { return new ElementIterator(); }
```

Code Fragment 7.14: Support for providing iterations of positions and elements of a LinkedPositionalList. (This should be nested within the LinkedPositionalList class definition of Code Fragments 7.9–7.12.)

www.EBooksWorld.ir